

1. Using a `List<Integer>`, return a new list containing only the even numbers.

Explanation: You use the `filter()` operation to select elements based on a condition, which checks if a number is divisible by 2. The `collect()` terminal operation gathers the results into a new List. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0) // Filter: Keep only numbers where the
    remainder is 0
    .collect(Collectors.toList());
// Result: [2, 4, 6]
```

2. Using Streams, how would you find the maximum value in a `List<Integer>`?

Explanation: You use the `max()` terminal operation, passing it a Comparator (here, the natural order). This returns an `Optional<Integer>` because the list might be empty. You then use `.get()` to access the value, or `.orElse()` for safety. **Solution:**

```
List<Integer> numbers = Arrays.asList(10, 5, 20, 15);
Optional<Integer> maxVal = numbers.stream()
    .max(Integer::compareTo); // Find the max element using natural order
// Result: Optional[20]
```

3. Using Streams, how would you calculate the sum of all elements in a `List<Integer>`?

Explanation: You use the `mapToInt()` intermediate operation to convert the stream of objects into a specialized primitive `IntStream`. The `IntStream` provides the efficient `sum()` terminal operation, which quickly calculates the total. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
int sum = numbers.stream()
    .mapToInt(Integer::intValue) // Convert to primitive IntStream
    .sum();                      // Calculate the sum efficiently
// Result: 10
```

4. Using Streams, how would you convert all strings to uppercase in a `List<String>`?

Explanation: You use the `map()` operation to transform each element. For every element, the operation applies the `String::toUpperCase` method, creating a new stream of uppercase strings. **Solution:**

```
List<String> words = Arrays.asList("hello", "world");
```

```
List<String> uppercased = words.stream()
    .map(String::toUpperCase) // Map: Apply the toUpperCase method to each
    element
    .collect(Collectors.toList());
// Result: ["HELLO", "WORLD"]
```

5. Using the Stream API, how would you sort a List<Integer> in ascending order?

Explanation: You use the **sorted()** intermediate operation. Calling sorted() without any arguments uses the natural order of the elements (ascending for integers). **Solution:**

```
List<Integer> numbers = Arrays.asList(5, 1, 4, 2);
List<Integer> sortedAsc = numbers.stream()
    .sorted() // Sorts using natural ascending order
    .collect(Collectors.toList());
// Result: [1, 2, 4, 5]
```

6. Using Streams, how would you count how many elements in a List<Integer> are greater than 5?

Explanation: You first use **filter()** to keep only the numbers that meet the condition (greater than 5). Then, you use the **count()** terminal operation to get the total number of elements that passed the filter. **Solution:**

```
List<Integer> numbers = Arrays.asList(3, 6, 8, 2, 7);
long count = numbers.stream()
    .filter(n -> n > 5) // Filter: Keep only elements greater than 5
    .count();          // Count the remaining elements
// Result: 3
```

7. Using Java Streams, how would you get a list of distinct elements from a List<Integer>?

Explanation: You use the **distinct()** intermediate operation. This operation uses the element's equals() and hashCode() methods to identify and filter out any duplicate elements, leaving only unique values. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 1, 4);
List<Integer> distinctNums = numbers.stream()
    .distinct() // Filters out duplicates
    .collect(Collectors.toList());
// Result: [1, 2, 3, 4]
```

8. Using `reduce()` in the Stream API, how would you reduce a `List<Integer>` to their sum?

Explanation: The `reduce()` operation combines all elements into a single result. You provide an identity (starting value, 0) and an accumulator (a function that adds two numbers).

Solution:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
int sum = numbers.stream()
    .reduce(0, (a, b) -> a + b); // Identity 0, Accumulator adds elements
// Result: 10
```

9. Using Streams, how would you return any one element from a `List<Integer>` (or an empty result if the list is empty)?

Explanation: You use the `findAny()` terminal operation. This returns an `Optional<Integer>`. It is useful for parallel streams because it can quickly return any element it finds, regardless of order. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3);
Optional<Integer> result = numbers.stream()
    .findAny(); // Returns one element non-deterministically
// Result: Optional[1] or Optional[2] or Optional[3]
```

10. Using Streams, how would you extract first names from a `List<String>` where each string is a full name like "John Doe"?

Explanation: You use the `map()` operation. For each full name, you apply a function that splits the string by the space character and takes only the first element (index 0). **Solution:**

```
List<String> fullNames = Arrays.asList("John Doe", "Jane Smith");
List<String> firstNames = fullNames.stream()
    .map(name -> name.split(" ")[0]) // Split by space and take the first
    part
    .collect(Collectors.toList());
// Result: ["John", "Jane"]
```

11. Using the Stream API, how would you check if all numbers in a `List<Integer>` are positive?

Explanation: You use the `allMatch()` terminal operation. This method returns true only if every single element in the stream satisfies the provided condition (being greater than 0). **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 5, 10);
boolean allPositive = numbers.stream()
```

```
.allMatch(n -> n > 0); // Checks if all elements are > 0  
// Result: true
```

12. Using Streams, how would you check that there are no negative numbers in a List<Integer>?

Explanation: You use the **noneMatch()** terminal operation. This method returns true if *no* element in the stream satisfies the condition (being less than 0). **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 0, 5);  
boolean noneNegative = numbers.stream()  
    .noneMatch(n -> n < 0); // Checks if no element is < 0  
// Result: true
```

13. Using Streams, how would you find the first element of a List<Integer> safely?

Explanation: You use the **findFirst()** terminal operation. This is safer than accessing the list by index because it returns an Optional<Integer>, which handles the case where the list might be empty. **Solution:**

```
List<Integer> numbers = Arrays.asList(10, 20, 30);  
Optional<Integer> firstElement = numbers.stream()  
    .findFirst(); // Returns Optional[10]
```

14. Using Streams and peek(), how would you print elements during stream processing without altering the result?

Explanation: You use the **peek()** intermediate operation. It allows you to perform an action (like printing or logging) on each element as it passes through the pipeline, without changing the element itself or the final result. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3);  
List<Integer> result = numbers.stream()  
    .peek(n -> System.out.println("Processing: " + n)) // Prints without  
    changing the stream  
    .collect(Collectors.toList());  
// Result: [1, 2, 3] (Prints 1, 2, 3 to console during execution)
```

15. Using Streams, how would you limit the output to the first 3 elements of a List<Integer>?

Explanation: You use the **limit(3)** intermediate operation. This operation truncates the stream, ensuring that only the first 3 elements of the stream are processed and passed downstream. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> limited = numbers.stream()
    .limit(3) // Limits the stream to the first 3 elements
    .collect(Collectors.toList());
// Result: [1, 2, 3]
```

16. Using Streams, how would you skip the first 2 elements of a List<Integer> and return the rest?

Explanation: You use the **skip(2)** intermediate operation. This operation discards the first two elements and then passes the remainder of the stream down the pipeline. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> skipped = numbers.stream()
    .skip(2) // Skips the first 2 elements (1 and 2)
    .collect(Collectors.toList());
// Result: [3, 4, 5]
```

17. Using the Stream API, how would you convert a List<Integer> to a Set<Integer> to remove duplicates?

Explanation: You use the **collect()** terminal operation with the **Collectors.toSet()** collector. Since a Set only allows unique elements, this automatically performs the deduplication process. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 2, 3);
Set<Integer> uniqueSet = numbers.stream()
    .collect(Collectors.toSet()); // Collects into a Set, automatically
    removing duplicates
// Result: {1, 2, 3}
```

18. Using IntStream and summaryStatistics(), how would you get summary statistics (min, max, average, count, sum) for a List<Integer>?

Explanation: You convert the stream to an **IntStream** using **mapToInt**. This specialized stream provides the **summaryStatistics()** method, which calculates all five metrics in a single pass. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
IntSummaryStatistics stats = numbers.stream()
    .mapToInt(Integer::intValue) // Convert to IntStream
    .summaryStatistics();        // Calculate all metrics
// Output: stats.getSum() returns 10, stats.getMax() returns 4
```

19. Using Stream functions, how would you find all even numbers from a List<Integer>?

Explanation: This is a duplicate of Q1. You use the **filter()** operation to select elements based on a condition, which checks if a number is divisible by 2. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0) // Selects numbers divisible by 2
    .collect(Collectors.toList());
// Result: [2, 4]
```

20. Using Streams, how would you get the first element from a List<Integer>?

Explanation: You use the **findFirst()** terminal operation. This returns an Optional<Integer>, which you can then check for presence before safely accessing the value. **Solution:**

```
List<Integer> numbers = Arrays.asList(10, 20, 30);
Optional<Integer> firstElement = numbers.stream()
    .findFirst();
// Result: Optional[10]
```

21. Using Streams, how would you count the total number of elements present in a List<Integer>?

Explanation: You use the **count()** terminal operation. This simple operation returns the total number of elements currently present in the stream pipeline. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
long totalCount = numbers.stream()
    .count();
// Result: 4
```

22. Using Streams, how would you find the maximum value element present in a List<Integer>?

Explanation: This is a duplicate of Q2. You use the **max()** terminal operation, passing it a Comparator (natural order) to find the largest value. **Solution:**

```
List<Integer> numbers = Arrays.asList(10, 5, 20);
Optional<Integer> maxVal = numbers.stream()
    .max(Integer::compareTo);
// Result: Optional[20]
```

23. Using Streams, how would you sort all values in a List<Integer> in ascending order?

Explanation: This is a duplicate of Q5. You use the **sorted()** intermediate operation without arguments, relying on natural ascending order. **Solution:**

```
List<Integer> numbers = Arrays.asList(5, 1, 4);
List<Integer> sortedAsc = numbers.stream()
    .sorted()
    .collect(Collectors.toList());
// Result: [1, 4, 5]
```

24. Using Streams, how would you sort all values in a List<Integer> in descending order?

Explanation: You use the **sorted()** intermediate operation and explicitly pass **Comparator.reverseOrder()** to change the natural sorting direction to descending. **Solution:**

```
List<Integer> numbers = Arrays.asList(5, 1, 4);
List<Integer> sortedDesc = numbers.stream()
    .sorted(Comparator.reverseOrder()) // Sorts from largest to smallest
    .collect(Collectors.toList());
// Result: [5, 4, 1]
```

25. Using Streams, how would you return true if any value appears at least twice in an int[] array, and false if all values are distinct?

Explanation: You convert the array to a stream, collect the elements into a Set (which removes duplicates), and compare the size of the set with the original array's length. If the lengths differ, duplicates existed. **Solution:**

```
int[] array = {1, 2, 3, 1};
boolean hasDuplicates = Arrays.stream(array)
    .distinct() // Remove duplicates
    .count() < array.length; // If count is less than original length,
    duplicates exist
// Result: true
```

26. Using the Java 8 Date and Time API together with Streams (if needed), how would you get and print the current date and time?

Explanation: You use the static `now()` method from the `LocalDateTime` class (or `ZonedDateTime` for timezone awareness) in the Java 8 Time API to capture the current system date and time. **Solution:**

```
LocalDateTime now = LocalDateTime.now(); // Captures current date and time
System.out.println(now);
```

27. Using Java 8 Streams, how would you concatenate two Streams into a single Stream?

Explanation: You use the static `Stream.concat()` method. This method takes two existing streams as input and returns a new stream that contains all elements from the first stream followed by all elements from the second stream. **Solution:**

```
Stream<Integer> s1 = Stream.of(1, 2);
Stream<Integer> s2 = Stream.of(3, 4);
Stream<Integer> combined = Stream.concat(s1, s2); // Combines both streams
// Result: [1, 2, 3, 4]
```

28. Using Java 8 Streams, how would you sort an array, and then convert the sorted array into a Stream?

Explanation: You use the built-in `Arrays.sort()` method on the array, and then use `Arrays.stream()` to convert the sorted array into a stream. **Solution:**

```
int[] array = {5, 1, 3};
Arrays.sort(array); // Sorts the array in place: [1, 3, 5]
IntStream sortedStream = Arrays.stream(array); // Converts to stream
// Result: Stream of [1, 3, 5]
```

29. Using `map()` in the Stream API, how would you convert all strings in a `List<String>` to uppercase?

Explanation: This is a duplicate of Q4. You use the `map()` operation to transform each element by calling `String::toUpperCase`. **Solution:**

```
List<String> words = Arrays.asList("a", "b");
List<String> uppercased = words.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
// Result: ["A", "B"]
```


30. Using Streams, how would you find the maximum element in an `int[]` or `List<Integer>`?

Explanation: This is a duplicate of Q2/Q22. For an `int[]` array, you use `Arrays.stream(array).max()`, which returns an `OptionalInt` and handles the primitive type efficiently. **Solution (using `int[]`):**

```
int[] array = {10, 5, 20};
OptionalInt maxVal = Arrays.stream(array)
    .max(); // Returns OptionalInt[20]
```

31. Using Streams, how would you print the count of each character in a `String`?

Explanation: You convert the string to a stream of characters, and use `Collectors.groupingBy()` to group the characters and `Collectors.counting()` to find the frequency of each group.

Solution:

```
String text = "hello";
Map<Character, Long> counts = text.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
// Result: {h=1, e=1, l=2, o=1}
```

32. Using the Stream API, how would you return the second-highest number from a `List<Integer>`?

Explanation: You use `sorted()` in reverse order, use `skip(1)` to skip the highest number, and then use `findFirst()` to retrieve the next element (the second highest). **Solution:**

```
List<Integer> numbers = Arrays.asList(10, 50, 20, 50);
Optional<Integer> secondHighest = numbers.stream()
    .distinct() // Handle potential duplicates of the highest number
    .sorted(Comparator.reverseOrder())
    .skip(1) // Skip the first (highest) element
    .findFirst();
// Result: Optional[20]
```

33. Using `Stream.max()` and a `Comparator`, how would you return the longest string from a `List<String>`?

Explanation: You use the `max()` terminal operation and provide a `Comparator` that compares the length of the two strings being passed to it. **Solution:**

```
List<String> words = Arrays.asList("a", "long", "word");
Optional<String> longest = words.stream()
```

```
.max(Comparator.comparing(String::length)); // Compares strings based
on their length
// Result: Optional["word"]
```

34. Using Streams, how would you remove all empty or blank strings from a List<String>?

Explanation: You use the **filter()** operation and apply the **String::isBlank** method in the negative. **isBlank()** checks for both empty strings and strings containing only whitespace.

Solution:

```
List<String> list = Arrays.asList("a", "", " ", "b");
List<String> cleaned = list.stream()
    .filter(s -> !s.isBlank()) // Filter: Keep only strings that are NOT
    blank
    .collect(Collectors.toList());
// Result: ["a", "b"]
```

35. Using Streams, how would you return the square of only the odd numbers from a List<Integer>?

Explanation: You use **filter()** to keep only the odd numbers ($n \% 2 \neq 0$). Then, you use **map()** to transform the remaining odd numbers by calculating their square ($n * n$). **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
List<Integer> squaredOdds = numbers.stream()
    .filter(n -> n \% 2 != 0) // Filter: Keep only odd numbers (1, 3)
    .map(n -> n * n) // Map: Calculate the square (1, 9)
    .collect(Collectors.toList());
// Result: [1, 9]
```

36. Using **mapToInt().average()**, how would you find the average of all numbers in a List<Integer>?

Explanation: You use **mapToInt()** to convert the stream to a primitive **IntStream**, which provides the **average()** method. This returns an **OptionalDouble** because the list might be empty. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
OptionalDouble average = numbers.stream()
    .mapToInt(Integer::intValue)
    .average(); // Calculates the average (2.5)
// Result: OptionalDouble[2.5]
```

37. Using Streams, how would you return only those strings whose length is greater than 5 from a List<String>?

Explanation: You use the **filter()** operation to keep only the strings whose length property is strictly greater than 5. **Solution:**

```
List<String> words = Arrays.asList("hello", "stream", "java");
List<String> longWords = words.stream()
    .filter(s -> s.length() > 5) // Filter: Keep only strings with length
    > 5
    .collect(Collectors.toList());
// Result: ["stream"]
```

38. Using Streams and a Comparator, how would you reverse-sort a List<String> ignoring case?

Explanation: You use the **sorted()** operation and pass **String.CASE_INSENSITIVE_ORDER** to handle case insensitivity. Then, you use **.reversed()** on that Comparator to achieve descending order. **Solution:**

```
List<String> words = Arrays.asList("Apple", "banana", "Cherry");
List<String> reverseSorted = words.stream()
    .sorted(String.CASE_INSENSITIVE_ORDER.reversed()) // Reverse sort,
    ignoring case
    .collect(Collectors.toList());
// Result: ["banana", "Cherry", "Apple"]
```

39. Using Streams and filter(), how would you remove elements from a list based on a predicate and collect the remaining ones into a new list?

Explanation: This is a general restatement of Q1/Q19/Q34. The technique is to define a **predicate** (a boolean function) inside the **filter()** operation. The filter keeps the elements that return true for the predicate, and **collect(Collectors.toList())** gathers the rest. **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 10, 20, 3);
List<Integer> remaining = numbers.stream()
    .filter(n -> n < 10) // Predicate: Keep only numbers less than 10
    .collect(Collectors.toList());
// Result: [1, 3]
```

40. Using `IntStream.rangeClosed`, Streams and `map`, how would you compute the sum of squares of the first 100 natural numbers?

Explanation: You use `IntStream.rangeClosed(1, 100)` to generate the numbers 1 through 100. Then, you use `map()` to square each number. Finally, you use `sum()` to calculate the total.

Solution:

```
long sumOfSquares = IntStream.rangeClosed(1, 100)
    .map(n -> n * n) // Square each number
    .sum();          // Calculate the final sum
```

JAVA Interview Buddy